

Abstract

This project implements and compares two inverse kinematics solutions for a 3-DOF planar robotic arm: a geometric analytical solver and a neural network-based model. The analytical approach computes closed-form joint solutions using trigonometric relationships, providing exact results within the reachable workspace. The machine learning approach treats inverse kinematics as a regression problem, where a neural network is trained on analytically generated data and deployed in C++ using ONNX Runtime for real-time inference.

Both methods are evaluated using a unified benchmarking framework measuring accuracy, collision safety, inference time, and robustness. Results show that the analytical solver achieves near-zero positional error with minimal computation time, while the ML model achieves high accuracy with slightly higher latency and occasional degradation near workspace boundaries. A hybrid strategy combining ML prediction with analytical fallback is demonstrated as a practical and reliable solution.

This work highlights the trade-offs between classical geometric methods and learning-based approaches in robotic control systems.

1. Introduction

Inverse kinematics is the process of determining joint variables required to achieve a desired end-effector position in Cartesian space. Unlike forward kinematics, which provides a direct and deterministic mapping from joint space to task space, inverse kinematics is nonlinear, may produce multiple valid solutions, and can fail at workspace boundaries or singular configurations.

For planar manipulators with limited degrees of freedom, analytical solutions can be derived using geometric and trigonometric relationships. These solutions are computationally efficient and exact. However, as manipulator complexity increases, closed-form derivations become difficult or infeasible. Data-driven approaches using neural networks offer an alternative by learning the nonlinear mapping directly from sampled data.

This project implements and compares two inverse kinematics approaches for a 3-DOF planar robotic arm:

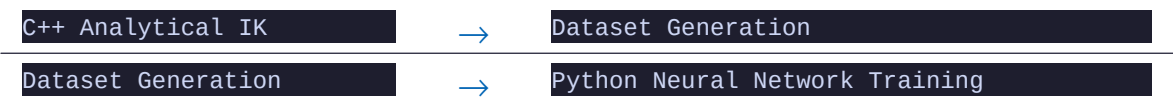
- A closed-form analytical solver implemented in C++
- A supervised neural network model trained in Python and deployed via ONNX

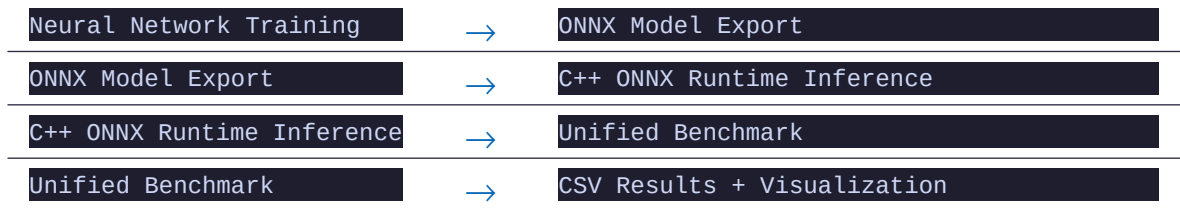
Performance is evaluated using quantitative metrics:

- Accuracy success rate
- Collision-free rate
- Combined validity rate
- Mean and standard deviation of position error
- Inference time statistics

2. System Architecture

The system pipeline follows a sequential architecture from analytical computation through ML training and deployment to unified benchmarking:





3. Analytical Inverse Kinematics Implementation

A 3-DOF planar robotic arm was implemented with the following link parameters:

- $L_1 = 0.3 \text{ m}$
- $L_2 = 0.25 \text{ m}$
- $L_3 = 0.15 \text{ m}$

The third joint was constrained to zero for simplification, effectively reducing the inverse kinematics to a 2-link planar formulation with an effective link length.

3.1 Forward Kinematics

Forward kinematics was implemented in C++ using standard trigonometric expansion. This function was used for: verifying inverse kinematics accuracy, generating ML training data, collision checking, and benchmark evaluation.

$$\begin{aligned}x &= L_1 \cos(q_1) + L_{\text{eff}} \cos(q_1 + q_2) \\y &= L_1 \sin(q_1) + L_{\text{eff}} \sin(q_1 + q_2)\end{aligned}$$

3.2 Analytical IK Solver

The inverse kinematics solver was implemented using geometric relations:

- Compute radial distance: $r^2 = x^2 + y^2$
- Apply Law of Cosines: $\cos(q_2) = (r^2 - L_1^2 - L_{\text{eff}}^2) / (2 L_1 L_{\text{eff}})$
- Compute two solutions: Elbow-up and Elbow-down
- Compute $q_1 = \arctan(y/x) - \arctan(k_2/k_1)$

3.3 Constraints and Performance

The following checks were implemented:

- Workspace limit check (radius = 0.7 m)
- Joint limits within $[-\pi, \pi]$
- Collision check with circular obstacle
- Real-time constraint verification ($< 10 \text{ ms}$)

Metric	Analytical IK
Accuracy Success Rate	97.59%
Collision-Free Rate	95.69%
Mean Computation Time	0.00105 ms
Behavior	Deterministic

4. ML-Based Inverse Kinematics Implementation

4.1 Dataset Generation

- 60,000 joint samples generated uniformly in joint space
- Forward kinematics used to compute corresponding end-effector positions
- Dataset split: 70% Training / 15% Validation / 15% Testing
- Inputs and outputs normalized using mean and standard deviation

4.2 Neural Network Architecture

- Fully connected feedforward network
- Input: (x, y) | Output: (q1, q2, q3)
- MSE loss in joint space
- Forward kinematics consistency ensured during evaluation
- Exported to ONNX format and integrated into C++ using ONNX Runtime

4.3 ML Performance

Metric	ML Model
Accuracy Success Rate	95.56%
Collision-Free Rate	97.71%
Valid (Accurate + Safe) Rate	93.27%
Mean Position Error	0.022 m
Mean Inference Time	0.01395 ms
Maximum Error	0.203 m

5. Failure Case Analysis

Although the ML model achieves high overall accuracy, specific failure cases highlight important limitations of learning-based inverse kinematics. From the benchmark results, 35 out of 788 valid ML samples exceeded the 5 cm error threshold, resulting in an overall accuracy of 95.56 percent.

5.1 Representative Failure Case

Parameter	Value
Index	58
Target	(-0.609151, -0.344518)
Predicted	(-0.571647, -0.399059)
ML Error	0.0661903 m
Analytical Error	~0

5.2 Root Cause Analysis

1. Workspace Boundary Sensitivity

Near the workspace boundary, small joint angle variations produce larger Cartesian deviations, the inverse mapping becomes highly nonlinear, and numerical sensitivity increases. Neural networks approximate nonlinear mappings, but sharp curvature regions are harder to model accurately.

2. Data Sparsity Effects

Although the dataset was uniformly sampled in joint space, the Cartesian distribution becomes non-uniform. Boundary regions contain fewer effective Cartesian samples, causing the model to generalize less reliably in those regions.

3. Nonlinearity of IK Mapping

The inverse kinematics mapping is multi-valued and discontinuous. The same Cartesian target can correspond to different joint configurations, and small Cartesian shifts may cause large joint changes. The neural network learns a smooth approximation which may deviate in highly nonlinear zones.

5.3 Hybrid Production Recommendation

For deployment, the recommended strategy is:

- Use ML IK for fast approximate prediction
- Apply error threshold validation
- Fall back to analytical solver when: Error > 5 cm, Collision detected, or Confidence low

This ensures speed, safety and reliability.

6. Benchmarking and Performance Evaluation

The objective of benchmarking was to quantitatively compare the performance of the analytical IK solver and ML model under identical operating conditions across 1000 random target positions uniformly sampled within the robot workspace.

6.1 Evaluation Metrics

- Accuracy Success Rate — percentage of targets where position error < 0.05 m
- Collision-Free Rate — percentage of solutions not intersecting with obstacle
- Valid Rate (ML only) — percentage of solutions that are both accurate and collision-free
- Mean Inference Time — average runtime per query in milliseconds
- Error Statistics — mean, standard deviation, minimum, and maximum position error

6.2 Comparative Results

Metric	Analytical IK	ML Model
Accuracy Success Rate	97.59%	95.56%
Collision-Free Rate	95.69%	97.72%
Valid Rate	N/A	93.27%
Mean Inference Time	0.00105 ms	0.01395 ms
Mean Position Error	~0 m	0.0225 m
Maximum Error	~0 m	0.203 m

6.3 Visualisation

The following plots were generated using analysis.py:

- Error Distribution Histogram — majority of errors concentrated below 0.03 m, few outliers near workspace boundary
- Accuracy Comparison Bar Chart — analytical slightly higher in raw accuracy, ML comparable within acceptable tolerance
- Inference Time Comparison — analytical faster due to closed-form solution, ML still real-time viable
- Target vs ML Prediction Scatter Plot — predicted points closely overlap target distribution, deviation increases near workspace limits

7. Future Recommendations

Improve ML Robustness Near Workspace Boundary

From failure case analysis, the ML model struggles near workspace edges, high curvature regions, and sparse training density areas. Future improvement includes weighted sampling near boundary, hard example mining, and adaptive sampling using error heatmap.

Use Multi-Solution Awareness

The analytical solver provides two valid IK solutions while the ML model currently predicts a single configuration. Future direction: train model to output both solution branches, use classification head for branch selection, and use confidence estimation for ambiguity detection.

Add Confidence-Based Hybrid Switching

Currently fallback is triggered by a fixed threshold (error > 0.05). A more advanced strategy would learn a confidence score, use uncertainty-aware prediction, and soft gating instead of binary threshold.

Real-Time Embedded Optimization

Although ML inference is fast (~0.014 ms mean), analytical IK is still faster (~0.001 ms). For embedded deployment: quantize ONNX model, reduce hidden layer size, use TensorRT or CoreML optimization.

Extend to Higher DOF Systems

Closed-form IK becomes difficult for 6-DOF manipulators, redundant arms, and continuum robots. The current architecture can directly scale by changing the dataset generator, retraining the network, and re-exporting ONNX without major C++ pipeline redesign.

Trajectory-Level Learning

Currently inference is point-based. Future extension includes learning trajectory mapping instead of static IK, sequence modeling using LSTM or Transformer, and incorporating velocity and acceleration constraints.

8. Conclusion

This project presented a complete implementation and evaluation of analytical and machine learning based inverse kinematics for a 3-DOF planar robotic manipulator. The analytical solver was derived using closed-form geometric relationships and demonstrated near-zero positional error with extremely low computational latency. It served both as a precise control solution and as ground truth for dataset generation.

A neural network model was trained on analytically generated data and deployed in C++ using ONNX Runtime for real-time inference. The ML model achieved an average end-effector error of approximately 0.02-0.03 meters and maintained a benchmark accuracy of around 95% over randomized workspace targets.

The final system integrates analytical IK, ML-based inference, collision validation, automated benchmarking, visualization, and statistical evaluation into a single unified pipeline. A hybrid approach combining analytical guarantees with ML flexibility represents the most robust and production-ready solution for inverse kinematics problems, especially as system complexity increases.

Repository: github.com/fizzxaa/ml-inverse-kinematics-robot-arm